K.S. INSTITUTE OF TECHNOLOGY, BANGALORE

Department of Computer Science & Design

DevOps Engineering Lab — BCSL657D — Lab Manual

Institution	K.S. Institute of Technology, Bangalore
Department	Computer Science & Design (CS&D)
Course	**DevOps Engineering Lab — BCSL657D**
Semester	VI | Academic Year 2025–26
Program No.	6 of 12
Program Title	Continuous Integration with Jenkins: Setting Up a CI Pipeline with Maven
Duration	~90 minutes (Concept: 10 min | Setup: 15 min | Pipeline: 25 min | Build+Demo: 25 min | Viva: 15 min)
Pre-requisite	Programs 1–5 completed — Jenkins running on port 8090, Maven3 registered in Tools
Maven Project	D:\Exercise\student-app (built and verified in Program 4)
Jenkins URL	http://localhost:8090 | Login: admin / admin123

1. The Journey So Far — Where We Are

Before starting, understand how Program 6 connects to everything you have done:

Program	Tool	What was done	What Program 6 uses from it
1	Maven + Gradle	Installed both tools, verified with hello-maven project	Maven 3.9.14 at C:\Program Files\Apache\maven — used by Jenkins to build
2	Maven Deep Dive	pom.xml, dependencies, commons-lang3, mvn clean install	pom.xml structure and mvn clean install command — Jenkins runs this exactly
3	Gradle Deep Dive	build.gradle, Version Catalog, gradle build	Not used directly — but concept of automated build is the same
4	Migration Exercise	student-app built with Maven — 2 tests, Student.java, App.java	student-app is the project Jenkins will build today — already verified working
5	Jenkins Setup	Jenkins 2.555.1 installed, Maven3 registered in Tools	Jenkins server running at localhost:8090, Maven3 ready to use
6	**THIS PROGRAM**	**Connect Jenkins to student-app — create CI pipeline**	**The culmination of all previous programs**

2. What This Program Is About

In Program 5 you installed Jenkins and explored the dashboard — like unboxing a robot. In Program 6 you WIRE that robot to your code. Jenkins will automatically build and test the student-app project every time you tell it to — and in a real company, it would do this on every code commit without any human intervention.

> **The Swiggy Kitchen Analogy:**
> Imagine Swiggy. Every time a chef changes a recipe, someone must manually test if the dish is correct. Now imagine 100 chefs changing 100 recipes every day — chaos! CI is like an automatic quality-check robot in the kitchen: every time a chef saves a change, the robot INSTANTLY tests the full dish and alerts if something is broken. Jenkins = the robot. Maven = the recipe rulebook. student-app = the dish being tested. Today, YOU wire the robot to the kitchen.

3. Learning Objectives

By the end of this lab, the student will be able to:

- Explain what Continuous Integration means and why it is used
- Create a Freestyle Job in Jenkins connected to a Maven project
- Configure the POM path and Maven version in a Jenkins build step
- Trigger a manual build and read the Console Output
- Interpret BUILD SUCCESS and BUILD FAILURE in Jenkins
- Intentionally break a test to see Jenkins catch the failure
- Fix the test and verify Jenkins returns to green

4. Pre-Requisites — Verify ALL Before Starting

Run every check below before creating any Jenkins job. Fixing issues mid-lab wastes 20 minutes.

Check 1 — Jenkins is running

```
REM If Jenkins is not already running, start it:
cd D:\Exercise\jenkins
java -jar jenkins.war --httpPort=8090

REM Wait for: Jenkins is fully up and running
REM Then open: http://localhost:8090
REM Login: admin / admin123
```

> ⚠ **Jenkins shows login screen instead of dashboard:**
> This is normal — Jenkins remembers the admin account from Program 5. Login with: Username: admin Password: admin123 The Unlock screen (initialAdminPassword) only appears on first-time setup. From now on, always use your admin credentials.

Check 2 — Maven is working

```
mvn -version

  Expected:
```

```
  Apache Maven 3.9.14
  Maven home: C:\Program Files\Apache\maven
  Java version: 21.0.10
```

Check 3 — student-app project exists and builds

```
cd D:\Exercise\student-app
mvn clean install

  Expected key lines:
  [INFO] Compiling 2 source files   ← Student.java + App.java
  [INFO] Tests run: 2, Failures: 0, Errors: 0
  [INFO] BUILD SUCCESS
```

⚠ If student-app does not exist or BUILD FAILURE:
Go back to Program 4 and recreate the student-app project. Do NOT proceed to Program 6 until mvn clean install gives BUILD SUCCESS with Tests run: 2, Failures: 0. Jenkins will fail for the exact same reason Maven fails manually. Always verify the manual build first.

Check 4 — Maven3 is registered in Jenkins

This was done in Program 5. Verify it is still there:

```
In browser: http://localhost:8090/manage/configureTools/

  Scroll to 'Maven installations'
  Expected: Name = Maven3, MAVEN_HOME = C:\Program Files\Apache\maven
  'Install automatically' checkbox must be UNCHECKED
```

If Maven3 is NOT there:
Go to Manage Jenkins → Tools → Maven installations → Add Maven. Name: Maven3 MAVEN_HOME: C:\Program Files\Apache\maven Uncheck 'Install automatically' → Click Save. This was done in Program 5 but may be missing if Jenkins was reinstalled or reset.

Observation — Confirm all 4 checks passed before proceeding:

5. Concepts in 10 Minutes — Teacher Speaks, Students Listen

Cover these three concepts before touching Jenkins. Total speaking time: 8–10 minutes.

Concept 1 — What is CI?

CI = Every time a developer saves (commits) their code, an automated system FETCHES that code, BUILDS it, TESTS it, and reports PASS or FAIL — without any human involvement.

The Google Classroom analogy:

Think of submitting an assignment on Google Classroom. The moment you submit, the system auto-checks: Is it on time? Is it the right format? Is it complete? And sends you a result instantly. Jenkins does the same for code. The moment a developer pushes code: Does it compile? Do all tests pass? Result in minutes, not days.

Concept 2 — Program 5 vs Program 6

Program 5 — Done	Program 6 — Today
Jenkins installed and running on port 8090	Jenkins pulls code, builds it, tests it automatically
Logged in, explored the dashboard	Create a Freestyle Job — Jenkins runs mvn clean install
Maven3 registered in Tools	Jenkins compiles Student.java + App.java, runs 2 tests
No code was built	See BUILD SUCCESS / FAILURE in Console Output

Concept 3 — The Three Actors in Today's Lab

Tool	Role	Analogy
Jenkins	The CI Server — the robot	Factory supervisor who kicks off the assembly line when a new part arrives
Maven	The Build Tool — runs compile + test	The assembly line — takes Java source code and outputs a working JAR
student-app	The code being built and tested	The product on the assembly line — Student Grade Report application

6. Part A — Verify the Maven Project (5 minutes)

We use student-app from Program 4. Do not create a new project. This saves time and reinforces continuity — Jenkins will build the same project students already built manually.

A1. Confirm the project structure

```
cd D:\Exercise\student-app
dir src\main\java\com\ksit

  Expected:
  App.java
  Student.java

dir src\test\java\com\ksit

  Expected:
  AppTest.java
```

A2. Confirm AppTest.java — JUnit 3 style

Open the test file to confirm it uses JUnit 3 (extends TestCase). This is important — it must match the JUnit version in pom.xml.

```
notepad src\test\java\com\ksit\AppTest.java

  Expected content:
  package com.ksit;
  import junit.framework.TestCase;

  public class AppTest extends TestCase {

      public void testGradeDistinction() {
          Student s = new Student("Test", 92);
          assertEquals("Distinction", s.getGrade());
      }

      public void testGradeFail() {
          Student s = new Student("Test", 35);
          assertEquals("Fail", s.getGrade());
      }
  }
```

Why JUnit 3 style (extends TestCase)?

The student-app pom.xml uses junit:junit:3.8.1 dependency. JUnit 3 requires tests to: 1. Extend TestCase 2. Method names must start with 'test' 3. assertEquals() is inherited from TestCase — no import needed This is different from JUnit 5 (@Test annotation) used in the Gradle project (Programs 3 and 4). Both styles work — they just belong to different JUnit versions.

A3. Run the manual build one final time to confirm

1 **Run mvn clean install from student-app folder**

This is the exact command Jenkins will run automatically

```
mvn clean install

  Watch for these lines — Jenkins will produce the SAME output:
  [INFO] Compiling 2 source files with javac [debug target 21]
  [INFO] Running com.ksit.AppTest
  [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
  [INFO] Building jar: target\student-app-1.0-SNAPSHOT.jar
  [INFO] BUILD SUCCESS
  [INFO] Total time: ~3 seconds
```

✓ Maven project verified!

student-app builds successfully with 2 passing tests. Jenkins will now automate exactly this process. Note the full path: D:\Exercise\student-app\pom.xml — you will need this in Part C.

Observation A — Write the path of pom.xml and confirm Tests run: 2, Failures: 0:

7. Part B — Verify Jenkins Setup (5 minutes)

Before creating the job, confirm Jenkins is correctly configured with Maven3. This was done in Program 5 — we are just verifying, not redoing it.

2 **Open Jenkins dashboard and log in**
URL: http://localhost:8090 | Username: admin | Password: admin123

3 **Verify Maven3 is in Tools**
Navigate to: Manage Jenkins (gear icon) → Tools → Scroll to Maven installations

```
Expected in Maven installations section:
  Name         : Maven3
  MAVEN_HOME   : C:\Program Files\Apache\maven
  Install automatically: unchecked
```

If Maven3 is missing — add it now:
Click 'Add Maven' → Name: Maven3 → MAVEN_HOME: C:\Program Files\Apache\maven → Uncheck 'Install automatically' → Save. Without this, the build step will fail with 'No Maven installation found'.

Observation B — Screenshot the Maven installations section showing Maven3:

8. Part C — Create the CI Freestyle Job (20 minutes)

What is a Freestyle Job?
A Freestyle Job is the simplest type of Jenkins job. It is configured entirely through the web UI — no scripting needed. You specify: where to get the code (SCM), what command to run (Build Step), and what to do after the build (Post-build actions). Perfect for learning CI concepts.

C1. Create a New Freestyle Job

4 **Click 'New Item' in the left sidebar of the Jenkins dashboard**
This is the starting point for creating any Jenkins job

```
Navigation: Jenkins Dashboard → New Item (left sidebar)

  Step 1: Enter item name: student-app-ci
  Step 2: Select: Freestyle project
  Step 3: Click OK

  You will be taken to the job configuration page.
```

Why the name 'student-app-ci'?
Good naming practice: the job name tells you which project it builds (student-app) and what it does (ci = continuous integration). In real companies, job names follow conventions like 'projectname-branch-type' e.g. 'payment-service-main-ci'.

C2. Configure General Settings

5 **In the General section — add a description**
Scroll to the top of the configuration page

```
Description field — type:
CI job for student-app Maven project.
Builds and tests the Student Grade Report application.
Connects to Programs 1-5 work at D:\Exercise\student-app
```

C3. Configure Source Code Management

6 **In the Source Code Management section — select None**
We are using a local project path, not a Git repository

In the SCM section you will see three options: None, Git, Subversion. Select None.

Why None instead of Git?
In a real company, Jenkins would pull code from GitHub/GitLab automatically. For this lab, we use the local project at D:\Exercise\student-app because: 1. Git setup is covered in later programs 2. The concept is identical — Jenkins runs mvn clean install regardless of where code comes from 3. Students can focus on the CI concept without Git complexity Tell students: 'The concept is identical — in your company job, this would point to GitHub.'

C4. Configure the Build Step — The Most Important Part

7 **Scroll to 'Build Steps' → Click 'Add build step'**
This is where you tell Jenkins WHAT to run

```
Build Steps section → Click 'Add build step' dropdown
  Select: 'Invoke top-level Maven targets'

  A Maven build configuration form appears.
```

8 **Configure the Maven build step with exact values**
Every field matters — wrong values will cause the build to fail

Field	Value to Enter
Maven Version	`Maven3  (select from dropdown — registered in Program 5)`
Goals	`clean install`
POM (under Advanced button)	`D:\Exercise\student-app\pom.xml`

How to enter the POM path:
In the Maven build step, click the 'Advanced' button to reveal the POM field. Enter the full path: D:\Exercise\student-app\pom.xml Jenkins needs the full absolute path because it does not know your working directory. If this field is left blank, Jenkins looks for pom.xml in the Jenkins workspace folder — and will fail because student-app is not there.

⚠ If Maven3 does not appear in the Maven Version dropdown:
Maven3 was not saved in Tools. Go to Manage Jenkins → Tools → Maven installations → Add Maven → Name: Maven3 → MAVEN_HOME: C:\Program Files\Apache\maven → Uncheck Install automatically → Save. Then come back to the job configuration.

C5. Save the Job

9	**Scroll to the bottom of the configuration page → Click Save** *The job is now created and visible on the Jenkins dashboard*

✓ Job created!
The job 'student-app-ci' is now visible on the Jenkins dashboard. The S (Status) and W (Weather) columns show a grey circle — indicating no builds have run yet. You are ready to trigger the first build.

9. Part D — Run the Build and Read the Output (20 minutes)

This is the most important part of Program 6. Read every line of the console output WITH the students — do not skip this.

D1. Trigger the First Build

10	**Click 'Build Now' in the left sidebar of the job page** *This manually triggers the build — simulating what would happen automatically on a Git commit*

```
Navigation: Click job name 'student-app-ci' → Build Now (left sidebar)

  Watch the Build History section (bottom left):
  A new entry appears: #1 with a spinning indicator
  The spinning indicator means the build is running

  Click on #1 → Click 'Console Output' in the left sidebar
```

D2. Read the Console Output — Line by Line with the Class

Point out each section of the console output and explain what it means:

```
Started by user admin                    ← Who triggered this build
Running as SYSTEM
Building in workspace C:\Users\lenovo\.jenkins\workspace\student-app-ci
                                         ← Jenkins creates a workspace
folder
```

```
[student-app-ci] $ cmd /c call
..\..\..\tools\hudson.tasks.Maven_MavenInstallation\Maven3\bin\mvn.cmd -f
D:\Exercise\student-app\pom.xml clean install
                                        ← Jenkins runs: mvn -f pom.xml
clean install

[INFO] Scanning for projects...
[INFO] Building student-app 1.0-SNAPSHOT
[INFO] --- compiler:3.11.0:compile ---
[INFO] Compiling 2 source files         ← Student.java + App.java
[INFO] --- surefire:test ---
[INFO] Running com.ksit.AppTest
[INFO] Tests run: 2, Failures: 0, Errors: 0   ← BOTH tests passed
[INFO] BUILD SUCCESS                    ← GREEN indicator
[INFO] Total time: 3.063 s
Finished: SUCCESS                       ← Jenkins marks this build as
SUCCESS
```

The key learning moment — ask the class:

Ask: 'What did Jenkins do that we did NOT do manually in Part A?' Answer: Exactly the same thing — it ran mvn clean install. The power is AUTOMATION. Jenkins can run this every 5 minutes, on every commit, for 100 developers simultaneously, forever — without a single human clicking anything.

Observation D1 — Write the workspace path Jenkins created and the total build time:

D3. Check the Build Result on the Dashboard

11 **Go back to the Jenkins dashboard**

Click 'Jenkins' in the breadcrumb at the top left

```
Expected on dashboard for student-app-ci job:

  S column: Blue circle  ← Build SUCCESS indicator
  W column: Sunny icon   ← Weather = all recent builds passed
  Last Success: Just now, Build #1
  Last Duration: ~3 seconds
```

Blue ball vs Green ball in Jenkins:

Older Jenkins versions show a blue ball for success (borrowed from Japanese traffic light convention where blue = go). Newer versions may show green. Both mean the same thing — BUILD SUCCESS. Red always means FAILURE.

10. Part E — Simulate a Failure and Fix It (15 minutes)

This is the most memorable part of the lab. Students see Jenkins catch a bug in real time — exactly as it would in a professional DevOps team.

E1. Break the Test Intentionally

12 **Edit AppTest.java to make testGradeFail() fail on purpose**
Change the assertion so it expects the wrong grade

```
notepad D:\Exercise\student-app\src\test\java\com\ksit\AppTest.java

  Find this method:
  public void testGradeFail() {
      Student s = new Student("Test", 35);
      assertEquals("Fail", s.getGrade());
  }

  Change ONE line — the assertEquals to expect wrong value:
  public void testGradeFail() {
      Student s = new Student("Test", 35);
      assertEquals("Distinction", s.getGrade());   ← WRONG — 35 marks =
Fail, not Distinction
  }

  Save the file.
```

Why this specific change?
A student with 35 marks gets 'Fail' from getGrade(). By telling assertEquals to expect 'Distinction', the test will fail because 'Distinction' does not equal 'Fail'. This simulates a real bug — a developer changed the grading logic but forgot to update the test.

E2. Trigger Build #2 and Observe the Failure

13 **Click Build Now again for student-app-ci job**
Build #2 will run — watch it turn RED

```
Expected console output for Build #2:

  [INFO] Running com.ksit.AppTest
  [ERROR] Tests run: 2, Failures: 1, Errors: 0
  [ERROR] testGradeFail(com.ksit.AppTest): expected:<[Distinction]> but
was:<[Fail]>
                                          ← Exact error message
  [INFO] BUILD FAILURE
  Finished: FAILURE                       ← Jenkins marks this as
FAILURE
```

⚠ What students see on the dashboard after Build #2:
S column changes to RED ball — failure indicator W column changes to stormy/rainy weather icon — recent builds failing Last Failure: Just now, Build #2 In a real company, Jenkins would now send an email alert to the developer who caused the failure. The build pipeline is blocked until the issue is fixed.

Observation E1 — Write the exact error message from the console output:

E3. Fix the Test and Return to Green

14 **Edit AppTest.java again — restore the correct assertion**
Fix the bug you intentionally introduced

```
notepad D:\Exercise\student-app\src\test\java\com\ksit\AppTest.java

  Restore the correct assertion:
  public void testGradeFail() {
      Student s = new Student("Test", 35);
      assertEquals("Fail", s.getGrade());   ← correct — back to original
  }

  Save the file.
```

15 **Click Build Now — trigger Build #3**
Watch the dashboard return to BLUE/GREEN

```
Expected console output for Build #3:

  [INFO] Tests run: 2, Failures: 0, Errors: 0
  [INFO] BUILD SUCCESS
  Finished: SUCCESS
```

✓ This is the complete CI experience!
Build #1 — SUCCESS (baseline) Build #2 — FAILURE (bug introduced — Jenkins caught it immediately) Build #3 — SUCCESS (bug fixed — Jenkins confirms it) This is exactly how CI works in every professional DevOps team. The difference in a real company: the bug is caught within minutes of the commit, not days later during integration testing.

Observation E2 — Write the build numbers and status for all 3 builds:

11. Expected Outputs — What to Record in Lab Journal

#	Output / Screenshot Required	What to Look For	Marks
1	Pre-requisites: mvn clean install output for student-app	Tests run: 2, Failures: 0, BUILD SUCCESS	Confirms project is ready
2	Jenkins dashboard showing job 'student-app-ci' created	Job visible with grey S/W indicators (no builds yet)	Job configuration saved correctly
3	Build #1 Console Output	BUILD SUCCESS, Tests run: 2, Failures: 0	Maven integrated with Jenkins

#	Output / Screenshot Required	What to Look For	Marks
4	Dashboard after Build #1	Blue/green ball in S column, sunny weather in W	SUCCESS state understood
5	Build #2 Console Output showing failure	Failures: 1, expected Distinction but was Fail, BUILD FAILURE	CI catches failures — core concept
6	Dashboard after Build #2	Red ball in S column, stormy weather in W column	FAILURE state understood
7	Build #3 Console Output after fix	Tests run: 2, Failures: 0, BUILD SUCCESS	Fix verified by CI
8	Build History showing all 3 builds	#1 blue, #2 red, #3 blue — clear pass-fail-pass sequence	Full CI cycle demonstrated

12. Student Observation Table — Copy into Lab Record

#	Build #	Status	Tests Run	Key Lines from Console
1	1			
2	2			
3	3			

13. Common Errors and Fixes

Error in Console Output	Root Cause	Fix
ERROR: 'Maven3' not found / No Maven installation found	Maven3 not registered in Jenkins Tools	Manage Jenkins → Tools → Add Maven → Name: Maven3 → MAVEN_HOME: C:\Program Files\Apache\maven → Save
ERROR: Cannot read D:\Exercise\student-app\pom.xml / No such file	Wrong POM path entered in build step	Click job → Configure → Build Steps → Advanced → correct the path to D:\Exercise\student-app\pom.xml
BUILD FAILURE: Tests run: 2, Failures: 1	AppTest.java has the intentional failure from Part E still applied	Restore assertEquals("Fail", s.getGrade()) in testGradeFail() and rebuild
BUILD FAILURE: cannot find symbol: Student	Student.java not in the project	Verify D:\Exercise\student-app\src\main\java\com\ksit\Student.java exists — recreate from Program 4 if missing
Finished: ABORTED instead of SUCCESS or FAILURE	Build was manually stopped	Click Build Now again — do not click the X button while a build is running
Job shows grey ball (never built) after saving	Build Now not clicked yet	Click the job name → Build Now in the left sidebar

Error in Console Output	Root Cause	Fix
Login screen appears instead of dashboard on startup	Jenkins session expired or new browser tab	Login again with admin / admin123

14. Viva-Voce Question Bank

Level 1 — Basic (Every student must answer these)

#	Question	Expected Answer
1	**What does CI stand for and what is its purpose?**	Continuous Integration — automatically building and testing code every time a developer commits a change, so bugs are caught within minutes instead of days or weeks.
2	**What is the role of Jenkins in this lab?**	Jenkins is the CI server — it reads the job configuration, runs mvn clean install on student-app, executes the 2 JUnit tests, and reports BUILD SUCCESS or BUILD FAILURE. It does this automatically without human involvement.
3	**What Maven command did Jenkins run? Where can you see this in the console output?**	mvn -f D:\Exercise\student-app\pom.xml clean install. It is visible in the console output as the line starting with '$ cmd /c call ..\mvn.cmd -f pom.xml clean install'.
4	**What did a RED ball on the Jenkins dashboard mean in Build #2?**	BUILD FAILURE — the test testGradeFail() failed because assertEquals expected 'Distinction' but getGrade() returned 'Fail' for a student with 35 marks.
5	**Why did we intentionally break the test in Part E?**	To demonstrate that Jenkins catches failures automatically — exactly as it would when a developer accidentally introduces a bug in real code. The test caught the wrong value in the assertEquals assertion.

Level 2 — Conceptual

#	Question	Expected Answer
1	**What is the difference between CI (Continuous Integration) and CD (Continuous Deployment)?**	CI only builds and tests — it verifies the code is correct. CD goes further and automatically deploys the tested code to a server or cloud environment. Today we did CI only — Program 8 covers deployment via Azure DevOps.
2	**Why must Jenkins know the full path D:\Exercise\student-app\pom.xml?**	Jenkins runs builds in its own workspace folder (C:\Users\lenovo\.jenkins\workspace\student-app-ci) — not in D:\Exercise\student-app. Without the full POM path, Maven cannot find the project. The -f flag tells Maven to use a specific pom.xml from any location.

#	Question	Expected Answer
3	**Build #2 failed. If this were a real company, what would happen next?**	Jenkins would send an email/Slack alert to the developer who last committed code. The failing build would block other team members from merging their changes. The developer must fix the bug and push a new commit to trigger a passing build before the pipeline can continue.
4	**What is the Jenkins workspace and what does it contain after a successful build?**	The workspace is at C:\Users\lenovo\.jenkins\workspace\student-app-ci. After a build, it contains whatever Maven produced — target\ folder with compiled classes, test results, and the JAR file. Jenkins builds in this isolated location to avoid affecting the original project folder.
5	**Could Jenkins trigger a build automatically on every code change? What would be needed?**	Yes. Instead of 'None' in Source Code Management, configure Git with the GitHub repository URL. Then enable 'Poll SCM' (check for changes on a schedule) or configure a GitHub webhook (push notification on every commit). Jenkins would then trigger automatically without clicking Build Now.

15. Quick Reference Card

Item	Detail
`Jenkins URL`	http://localhost:8090
`Admin login`	admin / admin123
`Maven project used`	D:\Exercise\student-app
`POM file path`	D:\Exercise\student-app\pom.xml
`Maven version in job`	Maven3 (registered in Program 5)
`Maven goals`	clean install
`Job name`	student-app-ci
`Job type`	Freestyle project
`Build trigger`	Manual — Build Now button
`Tests in project`	2 tests — testGradeDistinction() and testGradeFail()
`SUCCESS indicator`	Blue/green ball + 'Finished: SUCCESS' in console
`FAILURE indicator`	Red ball + 'Finished: FAILURE' in console
`Console output path`	Click job → Click build # → Console Output
`Build history location`	Left sidebar of job page
`Jenkins workspace`	C:\Users\lenovo\.jenkins\workspace\student-app-ci
`Jenkins home`	C:\Users\lenovo\.jenkins

— End of Program 6 Lab Manual —